

Documentación Base del Ecosistema Modular: Identity Gateway, Cartera, eFacturaEC y POS

1. Propósito del documento

Este documento define la arquitectura funcional y técnica inicial para construir un ecosistema modular de aplicaciones empresariales que puedan operar de forma independiente o integrada bajo una misma experiencia de autenticación, autorización y control comercial.

El objetivo principal es diseñar una base sólida para desarrollar productos SaaS independientes, inicialmente orientados a:

1. **Identity Gateway:** plataforma central de autenticación, autorización, SSO, federación y control de acceso.
2. **Cartera:** sistema de control de cuentas por cobrar, vencimientos, cobranza, seguimiento y reportes.
3. **eFacturaEC:** plataforma de facturación electrónica Ecuador con emisión, autorización, reportería y APIs.
4. **POS:** punto de venta integrado o independiente, con caja, ventas rápidas y conexión opcional con eFacturaEC.

Este documento servirá como insumo para generar prompts especializados de implementación para agentes IA de desarrollo, Cursor, Claude Code, Roo, Cline u otros entornos asistidos por IA.

2. Visión estratégica

La solución no debe construirse como una aplicación monolítica única, sino como una **suite modular SaaS**, donde cada producto pueda:

- Venderse de forma independiente.
- Operar con su propia lógica funcional.
- Compartir autenticación centralizada.
- Integrarse con otros productos de la suite.
- Integrarse con sistemas de terceros.
- Ser activado o desactivado por empresa, plan o contrato.
- Escalar técnica y comercialmente sin depender de todos los demás módulos.

La visión es crear una plataforma empresarial flexible que permita iniciar con una solución puntual, como Cartera o eFacturaEC, y luego ampliar el ecosistema hacia POS, reportería, integraciones, automatización e inteligencia artificial.

3. Principio rector del ecosistema

Cada producto debe poder vivir solo, pero todos deben poder compartir identidad, permisos, planes y acceso unificado.

Esto significa que eFacturaEC, POS y Cartera no deben depender funcionalmente uno del otro para operar, pero sí deben poder conectarse al Identity Gateway para autenticación, autorización, control de planes y acceso federado.

4. Productos iniciales del ecosistema

4.1 Identity Gateway

Servicio central encargado de:

- Login.
- Registro de empresas.
- Registro de usuarios.
- Roles y permisos.
- Módulos contratados.
- Planes y suscripciones.
- Tokens JWT.
- Refresh tokens.
- API Keys.
- SSO entre productos propios.
- Federación con sistemas externos.
- Auditoría de accesos.
- Seguridad centralizada.

4.2 Cartera

Producto para gestión de cuentas por cobrar.

Funciones esperadas:

- Registro de clientes.
- Registro de documentos por cobrar.
- Saldos pendientes.
- Vencimientos.
- Promesas de pago.
- Seguimiento de cobranza.
- Gestión de gestores o responsables.
- Reportes de cartera vencida.
- Reportes por cliente.
- Reportes por antigüedad de saldo.
- Integración con eFacturaEC para cargar facturas emitidas.
- Integración con terceros mediante API.

4.3 eFacturaEC

Producto para facturación electrónica Ecuador.

Funciones esperadas:

- Facturas.

- Retenciones.
- Notas de crédito.
- Notas de débito.
- Guías de remisión.
- Liquidaciones de compra.
- Firma electrónica.
- XML SRI.
- Autorización SRI.
- RIDE PDF.
- API de emisión para terceros.
- Reporterías tributarias.
- Control de paquetes de comprobantes.

4.4 POS

Producto para punto de venta.

Funciones esperadas:

- Apertura y cierre de caja.
- Venta rápida.
- Clientes.
- Productos.
- Formas de pago.
- Tickets.
- Facturación desde POS.
- Integración opcional con eFacturaEC.
- Control básico de inventario o conexión con inventario externo.
- Reportes de caja.

5. Modelo comercial esperado

La plataforma debe permitir vender productos de forma individual o empaquetada.

Ejemplos de planes:

Plan	Módulos incluidos	Usuarios	Límite comprobantes	Observación
eFactura Básico	eFacturaEC	2	500/mes	Solo emisión electrónica
Cartera Básico	Cartera	3	N/A	Solo cobranza
POS Básico	POS	2	N/A	Punto de venta independiente
Comercial Plus	eFacturaEC + Cartera	5	2.000/mes	Facturación y cobranza

Plan	Módulos incluidos	Usuarios	Límite comprobantes	Observación
Retail Suite	POS + eFacturaEC + Cartera	10	5.000/mes	Suite completa
Enterprise	Todos	Personalizado	Personalizado	Integraciones avanzadas

6. Arquitectura conceptual general

```

flowchart TD
    U[Usuario] --> IG[Identity Gateway]
    EXT[Sistema externo / ERP cliente] --> IG
    IG --> EF[eFacturaEC]
    IG --> POS[POS]
    IG --> CAR[Cartera]
    EF --> EFApi[API eFacturaEC]
    POS --> POSApi[API POS]
    CAR --> CARApi[API Cartera]
    IG --> AUTHDB[(PostgreSQL auth schema)]
    EFApi --> EFDB[(PostgreSQL efactura schema)]
    POSApi --> POSDB[(PostgreSQL pos schema)]
    CARApi --> CARDB[(PostgreSQL carterita schema)]
    IG --> BILL[(billing schema)]
    IG --> AUD[(audit schema)]

```

7. Decisión arquitectónica principal

7.1 No construir un monolito funcional

No se recomienda desarrollar una sola aplicación gigante que contenga facturación, POS y cartera en un mismo núcleo funcional rígido.

7.2 Construir un ecosistema modular

Se recomienda desarrollar productos separados, conectados mediante:

- Identity Gateway.
- Tokens JWT.
- OAuth2 / OpenID Connect.
- API Gateway opcional.
- Contratos REST.

- Eventos futuros mediante colas.
-

8. Identity Gateway

8.1 Descripción

Identity Gateway es el componente central del ecosistema. Su función es gestionar la identidad del usuario, empresas, permisos, módulos contratados y accesos federados.

No debe limitarse a ser una pantalla de login. Debe convertirse en la base de gobierno del ecosistema.

8.2 Responsabilidades

- Autenticación de usuarios.
- Emisión de tokens.
- Renovación de sesiones.
- Revocación de sesiones.
- Gestión de empresas.
- Gestión de usuarios.
- Gestión de roles.
- Gestión de permisos.
- Gestión de módulos contratados.
- Gestión de planes comerciales.
- Gestión de API Keys.
- Gestión de clientes terceros.
- Federación de identidad.
- Auditoría.
- Integración con productos internos.
- Integración con sistemas externos.

8.3 Capacidades mínimas

- Login con usuario y contraseña.
 - JWT Access Token.
 - Refresh Token.
 - Logout centralizado.
 - Cambio de empresa activa.
 - Cambio de módulo activo.
 - Consulta de permisos del usuario.
 - Consulta de módulos habilitados.
 - Token para navegación SSO.
 - Token federado para terceros.
 - API Keys para integraciones backend-to-backend.
-

9. Tipos de autenticación soportados

9.1 Login propio

El usuario existe directamente en la plataforma.

```
sequenceDiagram
    participant U as Usuario
    participant IG as Identity Gateway
    participant DB as Auth DB

    U->>IG: Usuario y contraseña
    IG->>DB: Validar credenciales
    DB-->>IG: Usuario válido
    IG-->>U: Access Token + Refresh Token
```

9.2 SSO entre productos propios

El usuario inicia sesión una vez y accede a eFacturaEC, POS o Cartera sin volver a ingresar credenciales.

```
sequenceDiagram
    participant U as Usuario
    participant IG as Identity Gateway
    participant EF as eFacturaEC
    participant CAR as Cartera

    U->>IG: Login
    IG-->>U: Token JWT
    U->>EF: Acceso con token
    EF->>IG: Validar token/permisos
    IG-->>EF: Acceso permitido
    U->>CAR: Abre Cartera
    CAR->>IG: Validar token/permisos
    IG-->>CAR: Acceso permitido
```

9.3 Login federado para terceros

Permite que un sistema externo autentique a sus usuarios y solicite un acceso seguro hacia un producto específico, como Cartera.

```
sequenceDiagram
    participant U as Usuario externo
    participant ERP as Sistema tercero
    participant IG as Identity Gateway
    participant CAR as Cartera

    U->>ERP: Ya está autenticado
    ERP->>IG: Solicita token federado
```

```
IG->>IG: Valida cliente, contrato y permisos
IG-->>ERP: Token federado temporal
ERP-->>U: Redirige a Cartera con token
U->>CAR: Ingresa con token federado
CAR->>IG: Valida token
IG-->>CAR: Acceso autorizado
```

9.4 API Keys para integraciones

Para servicios backend-to-backend donde no hay usuario interactivo.

Ejemplo:

- ERP cliente envía facturas a eFacturaEC.
- ERP cliente envía cartera a Cartera.
- Sistema externo consulta saldos.

10. Protocolo recomendado

10.1 Para productos propios

- OAuth2.
- OpenID Connect.
- JWT.
- Refresh Token.
- Scopes.
- Claims por empresa y módulo.

10.2 Para terceros

Soportar tres niveles:

1. **API Key simple** para integraciones servidor a servidor.
2. **OAuth2 Client Credentials** para integraciones formales.
3. **Authorization Code + PKCE / token federado** para acceso interactivo desde sistemas externos.

10.3 Para empresas grandes

Evaluar soporte futuro de:

- SAML 2.0.
- Azure AD / Entra ID.
- Google Workspace.
- LDAP empresarial.

No debe ser obligatorio en fase inicial.

11. Claims sugeridos en JWT

El token debe incluir información mínima y segura.

Ejemplo conceptual:

```
{
  "sub": "user-uuid",
  "tenant_id": "company-uuid",
  "company_ruc": "0999999999001",
  "active_module": "cartera",
  "modules": ["efactura", "cartera"],
  "roles": ["ADMIN_EMPRESA", "GESTOR_CARTERA"],
  "permissions": [
    "cartera.documentos.read",
    "cartera.documentos.create",
    "cartera.gestion.update"
  ],
  "token_type": "access",
  "iss": "identity-gateway",
  "aud": "veepra-suite",
  "exp": 1710000000
}
```

No se debe sobrecargar el token con información pesada. Para permisos complejos, el producto debe consultar al Identity Gateway o mantener caché temporal.

12. Modelo de permisos

12.1 Niveles de autorización

El sistema debe manejar permisos en varios niveles:

1. Usuario.
2. Empresa.
3. Producto/módulo.
4. Rol.
5. Permiso granular.
6. Plan contratado.
7. Reglas comerciales.

12.2 Ejemplo de permisos

Módulo	Permiso	Descripción
cartera	cartera.dashboard.view	Ver dashboard de cartera
cartera	cartera.documentos.read	Consultar documentos

Módulo	Permiso	Descripción
cartera	cartera.documentos.create	Registrar documentos
cartera	cartera.gestion.update	Registrar seguimiento
efactura	efactura.factura.emit	Emitir facturas
efactura	efactura.xml.download	Descargar XML
pos	pos.caja.open	Abrir caja
pos	pos.venta.create	Registrar venta

13. Modelo de datos sugerido PostgreSQL

13.1 Estrategia por esquemas

Usar PostgreSQL con esquemas separados:

```
auth.*
billing.*
efactura.*
pos.*
cartera.*
audit.*
integration.*
```

Ventajas:

- Permite separación lógica.
- Facilita mantenimiento inicial.
- Reduce complejidad operativa.
- Permite separación futura por base de datos.
- Permite consultas cruzadas controladas.

14. Esquema auth

Tablas principales:

```
auth.companies
auth.users
auth.user_credentials
auth.company_users
auth.roles
auth.permissions
auth.role_permissions
auth.user_roles
```

```
auth.modules
auth.company_modules
auth.sessions
auth.refresh_tokens
auth.api_clients
auth.api_keys
auth.federated_clients
auth.federated_tokens
auth.login_attempts
```

14.1 auth.companies

Campo	Tipo	Descripción
id	uuid	Identificador interno
ruc	varchar(13)	RUC de la empresa
legal_name	varchar	Razón social
commercial_name	varchar	Nombre comercial
email	varchar	Correo principal
status	varchar	ACTIVE, INACTIVE, SUSPENDED
created_at	timestamp	Fecha creación
updated_at	timestamp	Fecha actualización

14.2 auth.users

Campo	Tipo	Descripción
id	uuid	Identificador usuario
email	varchar	Correo único
username	varchar	Usuario
first_name	varchar	Nombres
last_name	varchar	Apellidos
status	varchar	ACTIVE, INACTIVE, BLOCKED
created_at	timestamp	Fecha creación
updated_at	timestamp	Fecha actualización

14.3 auth.company_users

Relación entre usuarios y empresas.

Campo	Tipo	Descripción
id	uuid	Identificador
company_id	uuid	Empresa
user_id	uuid	Usuario
status	varchar	ACTIVE, INACTIVE
default_company	boolean	Empresa por defecto
created_at	timestamp	Fecha creación

14.4 auth.modules

Campo	Tipo	Descripción
id	uuid	Identificador
code	varchar	efactura, pos, cartera
name	varchar	Nombre del módulo
description	text	Descripción
status	varchar	ACTIVE, INACTIVE

14.5 auth.company_modules

Define qué módulos tiene contratados una empresa.

Campo	Tipo	Descripción
id	uuid	Identificador
company_id	uuid	Empresa
module_id	uuid	Módulo
status	varchar	ACTIVE, SUSPENDED, CANCELLED
activation_date	date	Fecha activación
expiration_date	date	Fecha expiración

15. Esquema billing

Tablas sugeridas:

```

billing.plans
billing.plan_modules
billing.subscriptions
billing.package_limits

```

```
billing.consumption_records  
billing.invoices  
billing.payment_records
```

Debe permitir controlar:

- Usuarios permitidos.
- Módulos incluidos.
- Número de comprobantes.
- Consumo mensual.
- Servicios API.
- Integraciones activas.
- Estado comercial del cliente.

16. Esquema cartera

Tablas sugeridas:

```
cartera.customers  
cartera.documents_receivable  
cartera.document_payments  
cartera.collection_tasks  
cartera.collection_notes  
cartera.payment_promises  
cartera.aging_snapshots  
cartera.customer_risk_scores  
cartera.integration_imports  
cartera.report_snapshots
```

16.1 Funcionalidad principal de Cartera

- Cargar documentos por cobrar.
- Registrar pagos.
- Controlar vencimientos.
- Clasificar cartera por antigüedad.
- Registrar gestiones de cobranza.
- Crear compromisos de pago.
- Medir cumplimiento.
- Emitir reportes.
- Integrarse con eFacturaEC o sistemas externos.

17. Esquema efactura

Tablas sugeridas:

```
efactura.issuers
efactura.establishments
efactura.emission_points
efactura.sequences
efactura.digital_certificates
efactura.customers
efactura.products
efactura.documents
efactura.document_details
efactura.document_taxes
efactura.document_status_history
efactura.document_files
efactura.sri_logs
efactura.email_logs
efactura.api_emission_logs
efactura.custom_field_definitions
efactura.document_custom_values
efactura.business_rules
```

18. Esquema POS

Tablas sugeridas:

```
pos.stores
pos.cash_registers
pos.cash_sessions
pos.products
pos.product_prices
pos.sales
pos.sale_details
pos.payments
pos.sale_tickets
pos.cash_movements
pos.pos_settings
pos.device_agents
```

19. Esquema audit

Tablas sugeridas:

```
audit.access_logs
audit.security_events
audit.business_events
audit.api_request_logs
```

```
audit.data_change_logs  
audit.error_logs
```

Debe registrar:

- Quién accedió.
- Desde dónde.
- A qué empresa.
- A qué módulo.
- Qué acción ejecutó.
- Resultado.
- Fecha y hora.
- IP.
- User agent.

20. Esquema integration

Tablas sugeridas:

```
integration.external_systems  
integration.webhooks  
integration.webhook_events  
integration.integration_tokens  
integration.inbound_messages  
integration.outbound_messages  
integration.integration_errors
```

21. Estrategia de independencia de productos

Cada producto debe tener:

- Su propio frontend o módulo Angular desacoplado.
- Su propio API backend.
- Su propio esquema de base de datos.
- Su propio dominio funcional.
- Sus propios reportes.
- Sus propias reglas de negocio.

Pero debe compartir:

- Identity Gateway.
- Empresas.
- Usuarios.
- Permisos.
- Planes.
- Auditoría base.

- Contratos de integración.
-

22. Posibles modelos de despliegue

22.1 Modelo inicial recomendado

Un repositorio por producto o monorepo organizado.

Backend:

```
identity-service
cartera-service
efactura-service
pos-service
```

Frontend:

```
identity-web
cartera-web
efactura-web
pos-web
suite-shell-web
```

22.2 Modelo alternativo inicial más simple

Monorepo:

```
/apps
  /identity-api
  /cartera-api
  /efactura-api
  /pos-api
  /suite-web
/libs
  /shared-auth
  /shared-ui
  /shared-contracts
  /shared-utils
```

23. Frontend Angular

23.1 Principio de diseño

Usar Angular con Atomic Design y una experiencia visual inspirada en Hope UI.

Referencia visual:

<https://templates.iqonic.design/hope-ui/html/dist/dashboard/>

23.2 Reglas visuales

- Dashboard moderno.
- Sidebar lateral.
- Navbar superior.
- Cards de métricas.
- Widgets estadísticos.
- Tablas administrativas.
- Formularios limpios.
- Pantallas responsivas.
- Soporte futuro de modo claro/oscuro.
- Estética SaaS profesional.

23.3 Atomic Design

```
atoms/  
  buttons/  
  inputs/  
  labels/  
  badges/  
  icons/  
  loaders/  
  
molecules/  
  search-box/  
  form-field/  
  metric-card/  
  filter-bar/  
  action-menu/  
  
organisms/  
  data-grid/  
  sidebar-menu/  
  top-navbar/  
  module-launcher/  
  login-panel/  
  permissions-matrix/  
  
templates/  
  auth-layout/  
  dashboard-layout/  
  module-layout/  
  public-layout/  
  
pages/
```



```
login/  
dashboard/  
companies/  
users/  
modules/  
cartera/  
efactura/  
pos/
```

24. Suite Shell

Se recomienda crear una aplicación principal tipo portal:

```
suite-shell-web
```

Responsabilidades:

- Login.
- Selección de empresa.
- Selección de módulo.
- Visualización de productos contratados.
- Redirección SSO.
- Perfil de usuario.
- Configuración general.

25. Flujo de acceso a módulo

```
flowchart TD  
  A[Usuario inicia sesión] --> B[Identity Gateway valida credenciales]  
  B --> C[Obtiene empresas asociadas]  
  C --> D[Usuario selecciona empresa]  
  D --> E[Identity obtiene módulos contratados]  
  E --> F[Suite Shell muestra módulos disponibles]  
  F --> G{Usuario elige módulo}  
  G --> H[eFacturaEC]  
  G --> I[POS]  
  G --> J[Cartera]  
  H --> K[Validación de token y permisos]  
  I --> K  
  J --> K
```

26. Integración entre Cartera y eFacturaEC

26.1 Escenario

Cuando eFacturaEC emite una factura autorizada, esa factura puede alimentar automáticamente Cartera como documento por cobrar.

26.2 Modelo sincrónico inicial

```
sequenceDiagram
    participant EF as eFacturaEC
    participant CAR as Cartera
    participant IG as Identity Gateway

    EF->>IG: Validar permiso de integración
    IG-->>EF: Integración permitida
    EF->>CAR: Crear documento por cobrar
    CAR-->>EF: Documento cartera creado
```

26.3 Modelo futuro con eventos

```
flowchart LR
    EF[eFacturaEC] --> EVT[Evento: factura_autorizada]
    EVT --> MQ[Cola/Event Bus]
    MQ --> CAR[Cartera crea documento por cobrar]
```

27. Integración con terceros

27.1 Caso de uso

Un ERP externo desea usar solo el módulo Cartera. Sus usuarios ya están autenticados en su plataforma, por lo que no deben volver a ingresar usuario y contraseña.

27.2 Flujo conceptual

1. El cliente registra su sistema externo en Identity Gateway.
 2. Identity Gateway entrega client_id y client_secret.
 3. El sistema externo solicita un token federado.
 4. Identity Gateway valida contrato, empresa y módulo.
 5. El usuario es redirigido a Cartera con token temporal.
 6. Cartera valida token y abre sesión funcional.
-

28. Endpoints iniciales Identity Gateway

28.1 Autenticación

```
POST /auth/login
POST /auth/refresh
POST /auth/logout
GET /auth/me
GET /auth/me/companies
GET /auth/me/modules
GET /auth/me/permissions
```

28.2 Empresas

```
GET /companies
POST /companies
GET /companies/{id}
PUT /companies/{id}
PATCH /companies/{id}/status
```

28.3 Usuarios

```
GET /users
POST /users
GET /users/{id}
PUT /users/{id}
PATCH /users/{id}/status
POST /users/{id}/companies
POST /users/{id}/roles
```

28.4 Módulos

```
GET /modules
POST /companies/{companyId}/modules/{moduleId}/activate
PATCH /companies/{companyId}/modules/{moduleId}/suspend
```

28.5 Federación

```
POST /federation/clients
POST /federation/token
POST /federation/exchange
GET /federation/validate
```

28.6 API Keys

```
POST    /api-keys
GET      /api-keys
PATCH  /api-keys/{id}/revoke
POST    /api-keys/{id}/rotate
```

29. Endpoints iniciales Cartera

```
GET      /cartera/dashboard
GET      /cartera/customers
POST     /cartera/customers
GET      /cartera/documents
POST     /cartera/documents
GET      /cartera/documents/{id}
POST     /cartera/documents/{id}/payments
POST     /cartera/documents/{id}/collection-notes
POST     /cartera/documents/{id}/payment-promises
GET      /cartera/reports/aging
GET      /cartera/reports/customer-balance
GET      /cartera/reports/overdue
```

30. Seguridad

30.1 Reglas generales

- Contraseñas con hash seguro.
- JWT firmado.
- Refresh tokens persistidos y revocables.
- Tokens con expiración corta.
- Rate limiting en login.
- Bloqueo por intentos fallidos.
- Auditoría de accesos.
- Validación de permisos por endpoint.
- Separación por tenant/empresa.
- API Keys hasheadas.
- Secretos cifrados.
- CORS restringido.

30.2 Seguridad en tokens federados

- Token de corta duración.
- Firmado por Identity Gateway.
- Un solo uso si aplica.
- Validación de client_id.

- Validación de redirect_uri.
 - Validación de módulo permitido.
 - Registro de auditoría.
-

31. Backend Spring Boot

31.1 Estructura base Identity Service

```
com.veepra.identity
  config
  security
  auth
  company
  user
  role
  permission
  module
  billing
  federation
  apikey
  audit
  common
```

31.2 Estructura base Cartera Service

```
com.veepra.cartera
  config
  security
  tenant
  customer
  document
  payment
  collection
  promise
  report
  integration
  audit
  common
```

32. Estrategia de multitenancy

32.1 Recomendación inicial

Usar multitenancy por columna:

```
company_id uuid not null
```

En todas las tablas transaccionales.

Ventajas:

- Más simple para iniciar.
- Mejor para SaaS multiempresa.
- Menor costo operativo.
- Fácil reportería centralizada.
- Compatible con separación futura.

32.2 Reglas obligatorias

- Todo query de datos de negocio debe filtrar por `company_id`.
- El `company_id` debe venir del token o contexto de sesión, no del frontend libremente.
- Crear `TenantContext` en backend.
- Validar que el usuario pertenece a la empresa activa.
- Evitar acceso cruzado entre empresas.

33. Contexto de sesión

Cada producto debe resolver un contexto común:

```
user_id  
company_id  
company_ruc  
active_module  
roles  
permissions  
plan  
subscription_status
```

Este contexto debe estar disponible en los servicios backend.

34. Roadmap recomendado

Fase 1: Identity Gateway base

- Login.
- Usuarios.
- Empresas.
- Roles.
- Permisos.
- Módulos.
- JWT.

- Refresh token.
- Suite Shell básica.

Fase 2: Cartera base

- Clientes.
- Documentos por cobrar.
- Pagos.
- Vencimientos.
- Dashboard.
- Reporte de antigüedad.

Fase 3: SSO Identity + Cartera

- Validación de token.
- Permisos por módulo.
- Acceso desde Suite Shell.
- Logout centralizado.

Fase 4: Federación para terceros

- Registro de clientes externos.
- Client ID / Secret.
- Token federado.
- Redirección segura.
- Auditoría.

Fase 5: Integración con eFacturaEC

- Crear documento de cartera desde factura autorizada.
- Consultar facturas pendientes.
- Sincronizar clientes.
- Sincronizar pagos si aplica.

Fase 6: eFacturaEC completo

- Emisión electrónica.
- XML.
- Firma.
- SRI.
- RIDE.
- API pública.

Fase 7: POS

- Caja.
 - Venta.
 - Ticket.
 - Integración con eFacturaEC.
 - Integración con Cartera si hay crédito.
-

35. Riesgos técnicos

Riesgo	Impacto	Mitigación
Sobrediseñar desde el inicio	Alto	Implementar por fases, iniciar con Identity + Cartera
Mezclar lógica de productos	Alto	Separar servicios y esquemas
Tokens con demasiada información	Medio	Usar claims mínimos y consulta de permisos
Acceso cruzado entre empresas	Crítico	TenantContext obligatorio y pruebas de seguridad
Integración externa insegura	Alto	OAuth2, tokens cortos, client_secret, redirect_uri
Complejidad comercial de planes	Medio	Diseñar billing modular desde fase inicial
Dependencia fuerte entre eFactura y Cartera	Alto	Integrar por API/eventos, no por tablas compartidas
Cambios futuros de marca	Bajo	Usar naming configurable y no hardcodear marca

36. Decisiones recomendadas para iniciar

1. Construir primero Identity Gateway.
2. Construir Cartera como primer producto funcional.
3. Usar PostgreSQL con esquemas separados.
4. Usar Spring Boot para APIs.
5. Usar Angular con Atomic Design.
6. Usar JWT + Refresh Token.
7. Preparar OAuth2/OpenID Connect progresivamente.
8. Iniciar con multitenancy por company_id.
9. Separar desde el inicio módulos, permisos y planes.
10. Integrar eFacturaEC después de validar el flujo de identidad.

37. Definición de éxito de la primera etapa

La primera etapa será exitosa cuando:

- Un usuario pueda iniciar sesión en Identity Gateway.
- El usuario pueda seleccionar una empresa.
- El usuario pueda ver módulos contratados.
- El usuario pueda abrir Cartera sin nuevo login.
- Cartera valide token y permisos.
- Cartera filtre datos por empresa.
- Se pueda registrar un documento por cobrar.
- Se pueda consultar cartera vencida.

- Se pueda auditar el acceso.

38. Prompt maestro futuro para implementación

Actúa como arquitecto senior y desarrollador full stack experto en SaaS modular, Spring Boot, Angular, PostgreSQL, OAuth2/OpenID Connect, JWT, multitenancy y arquitectura de productos independientes.

Vamos a implementar un ecosistema modular compuesto por:

1. Identity Gateway.
2. Cartera.
3. eFacturaEC.
4. POS.

Cada producto debe poder funcionar de forma independiente, pero también debe integrarse en una suite con autenticación única.

Primera etapa:

- Diseñar e implementar Identity Gateway.
- Diseñar e implementar Cartera.
- Permitir SSO entre Suite Shell y Cartera.
- Preparar federación para terceros.

Restricciones:

- Backend con Java Spring Boot.
- Frontend con Angular.
- Base de datos PostgreSQL.
- Angular con Atomic Design.
- UI inspirada en Hope UI.
- Multitenancy por company_id.
- Seguridad con JWT y Refresh Token.
- Arquitectura preparada para OAuth2/OpenID Connect.
- Esquemas PostgreSQL separados: auth, billing, cartera, efactura, pos, audit, integration.

Genera:

1. Estructura de repositorios.
2. Modelo de base de datos.
3. Scripts SQL iniciales.
4. Entidades Java.
5. DTOs.
6. Repositories.
7. Services.
8. Controllers.
9. Seguridad JWT.
10. TenantContext.
11. Guards e interceptors Angular.
12. Componentes Atomic Design.
13. Pantallas principales.

14. Endpoints.
15. Pruebas.
16. Roadmap técnico por sprint.

No mezcles lógica de productos. Identity debe autenticar y autorizar; Cartera debe gestionar cuentas por cobrar. La comunicación entre productos debe hacerse por tokens, APIs y contratos claros.

39. Próximo paso recomendado

El siguiente documento debe profundizar exclusivamente en:

Identity Gateway - Diseño Técnico Detallado

Debe incluir:

- Modelo de datos definitivo.
- Flujo JWT.
- Flujo refresh token.
- Flujo cambio de empresa.
- Flujo activación de módulos.
- Flujo federado externo.
- Endpoints.
- Seguridad.
- Estructura Spring Boot.
- Estructura Angular.
- Primer backlog técnico.